

## 2. Use a small serverless proxy.

If you don't want to copy the library, we can use a Cloudflare Worker or Netlify proxy that fetches the file and adds the correct CORS headers.

(A bit more setup, but cleaner if the library changes often.)

## 3. Wait for Reown to publish a browser-friendly build.

Sometimes the docs show a URL like `appkit.umd.js` or `appkit.min.js` that is allowed in browsers.

We could check their docs for that version.

---

## Quick next step

Let's try the simplest route — **local copy** inside your repo:

1. Download that file I mentioned.
2. In your GitHub repo click **Add file** → **Upload files** and drag the saved `appkit-index.js`.
3. Edit your `index.html` to change the import line to:  

```
import { AppKit } from "../appkit-index.js";
```
4. Commit the change.
5. Refresh your GitHub Pages site after a minute or two.
6. Check the console again.

If that works, you'll see the CORS error disappear, and either a new "connection started" message or a different, smaller error that tells us we're finally executing the library code.

---

Once we have this first import running, we'll be past the worst of the "wild goose chase."  
After that, everything else is just wiring buttons and data flow—the fun part.

Ok Poof      before we do what you suggested, I want to tell you a few things. Unfortunately, I had to wait over an hour due to ChatGPT limits, so not sure how much code we will be able to share before we get cut off again, but in the JavaScript Reown AppKit documentation it says to create a config file to establish a singleton instance for the Universal Connector and provides code of an example using SUI and STACKS networks from a VITE project, but it starts with: `import { UniversalConnector }`

from '@reown/appkit-universal-connector' and returns universalConnector Then it says in the main.js file add: `import { getUniversalConnector } from './config/appKit.js'` async function `setup() { const universalConnector = await getUniversalConnector() // check if session is already connected if (universalConnector?.provider.session) { session = universalConnector?.provider.session } } setup()` So maybe if we configure our code to use the Universal Connector .js code this can work. But I also realized that I had forgotten to whitelist the domain: <https://legacygold.github.io/portalx-lite-test/>. When I added it, it said that it could take six hours to take effect. So I'm not sure if we should do what you suggested in your last response or try to work with what I just shared or both.

That's really good sleuthing, Debi —you're catching on to the same detail that tripped up most developers who tried Reown early on.

Let's translate what you just found into clear choices.

---

## What the “Universal Connector” actually does

- **UniversalConnector** is a helper module Reown created so one script can handle many different chains (Stellar, Sui, Stacks, etc.) without you writing a separate connection routine for each.
- The idea is that your `config/appKit.js` file sets it up once, and then any page can “import” it with 

```
import { getUniversalConnector } from './config/appKit.js'
```

and get back a single, already-initialized connector object.
- That pattern works beautifully **inside bundler-based projects** like Vite, Webpack, or Rollup, because those tools stitch all the modules together for you.

---

## Why it won't yet run in our current GitHub-Pages test

GitHub Pages is just serving plain HTML + JS.

It doesn't bundle imports from packages such as

`'@reown/appkit-universal-connector'`.

When the browser sees that line, it immediately fails with

Failed to resolve module specifier '@reown/appkit-universal-connector'.

So even if your domain whitelisting finishes propagating, the browser will still hit that import error unless a build step (like Vite) rewrites it.

---

## The practical paths forward

| Path                                         | What it involves                                                                                                                                                                                            | Pros                                                                    | Cons                                                     |
|----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------|
| <b>A – Stay in “no-build” mode (current)</b> | Keep one HTML file, copy the needed .js files manually into the repo.                                                                                                                                       | Simple, visible, easy to debug.                                         | You have to download any extra dependency yourself.      |
| <b>B – Adopt the Vite setup</b>              | Install Node.js on your computer, run <code>npm create vite@latest</code> , install <code>@reown/appkit</code> and <code>@reown/appkit-universal-connector</code> , then build and publish to GitHub Pages. | Full modern developer workflow, supports imports exactly like the docs. | More moving parts; requires command-line and Node tools. |

For now, I’d stay with **Path A** until we confirm that a connection can even open; once that’s solid we can always shift the project into a Vite workspace later.

---

### What to do right now

1. Let the Reown domain-whitelist update finish (give it the 6 hours).
2. In the meantime, try the **local-copy** fix I described earlier.
  - Download the file at <https://unpkg.com/@reown/appkit@1.8.14/dist/index.js>
    - save it as `appkit-index.js`
    - upload it beside your `index.html`.

- Change the import line in your HTML to

```
import { AppKit } from "./appkit-index.js";
```

3. When the whitelist window has passed, refresh your GitHub Pages URL and test again.

If that finally loads without a CORS error, we’ll know the environment can execute modules, and we can *then* decide whether to introduce the Universal Connector pattern or keep things lighter.

---

So in short:

-  Wait for whitelist propagation.

✅ Copy the library locally (removes CORS).

⌚ Post the new console output here once you've done those two things.

From there we'll either celebrate a working handshake or apply the smallest next patch instead of another wild goose chase.

Unfortunately Poof I forgot to tell you that the link:

<https://unpkg.com/@reown/appkit@1.8.14/dist/index.js> gets an error: Not found:

[/@reown/appkit@1.8.14/dist/index.js](#) So that was why I went looking for other methods. Reown's JavaScript AppKit documentation actually recommends using VITE for getting started with AppKit JavaScript, but when I went to the VITE site it went over my non-programmer head, so I looked for other options, but if you think that this is how real developers would do this, then perhaps that's what we should do, but you will need to walk me through how to do it. I have generally used VS Code when doing command line coding in the past back when I used ChatGPT to help me build the PortalX trading bot in Python for Coinbase Advanced trade. That was much more complicated coding, but back then I was not being cut off by ChatGPT limits. This was pre-IC Mode and before there were data limits on free ChatGPT accounts. So if you can explain to me how to use VS Code and VITE to build what we need to build and make appropriate commits to my GitHub (even though I really don't know what I am doing) then maybe we can get this set up so that it will work for the real app later.

Alright, Debi — you're thinking like a real project lead now.

If Reown themselves say “start with Vite,” then using Vite is the developer-standard route. It's not mystical—just a tiny helper that bundles the modules for the browser so you don't hit the CORS and “import” errors anymore.

Let's keep it as simple and linear as possible.

You'll only do six commands and a few button clicks.



## Step-by-step: create a minimal Vite + Reown AppKit project

### 1 Install the basic tools (one-time setup)

- **Node JS:** go to <https://nodejs.org> → download the “LTS” version → install it.  
That gives you the command `npm`.
- **VS Code:** you already have it, good.

After installing Node, open a terminal (in VS Code choose **View** → **Terminal**) and type:

```
node -v
```

```
npm -v
```

Both should print version numbers. If they do, we're ready.

---

## 2 Create a new Vite project folder

In the terminal:

```
npm create vite@latest portalx-lite
```

Vite will ask:

- Project name → just press Enter
- Framework → choose **Vanilla**
- Variant → choose **JavaScript**

Then:

```
cd portalx-lite
```

```
npm install
```

Now you have a fresh mini-site that works in any browser.

---

## 3 Install Reown packages

Inside that same folder:

```
npm install @reown/appkit @reown/appkit-universal-connector
```

---

## 4 Add the AppKit setup code

In VS Code open `src/main.js` and replace its contents with:

```
import { AppKit } from "@reown/appkit";  
  
import { UniversalConnector } from "@reown/appkit-universal-connector";
```

```

async function setup() {
  const universalConnector = new UniversalConnector({
    projectId: "YOUR_PROJECT_ID",
    wallets: [
      {
        id: "76a3d548a08cf402f5c7d021f24fd2881d767084b387a5325df88bc3d4b6f21b",
        name: "Lobstr"
      }
    ],
    chains: ["stellar:pubnet"]
  });

  // Wait until AppKit finishes loading
  const appKit = new AppKit({ universalConnector });

  const connectBtn = document.getElementById("connectBtn");
  connectBtn.addEventListener("click", async () => {
    try {
      const session = await appKit.connect({
        requiredNamespaces: {
          stellar: {
            chains: ["stellar:pubnet"],
            methods: ["stellar_signXDR", "stellar_signAndSubmitXDR"],
            events: ["chainChanged", "accountsChanged"]
          }
        }
      }
    }
  });
}

```

```

    });

    const account = session.namespaces.stellar.accounts[0];

    alert("Connected!\nPublic Key: " + account);

    console.log("Session object:", session);

  } catch (err) {

    console.error("Connection failed:", err);

    alert("Connection cancelled or failed.");

  }

});

}

```

setup();

Then open `index.html` in the project root and make sure it has a button with that ID:

```
<button id="connectBtn">Connect Lobstr</button>
```

---

## 5 Run it locally

In the terminal:

```
npm run dev
```

Vite will print something like:

```
Local: http://localhost:5173/
```

Open that link in Brave or Chrome.

You should see your button, and when you click it, the WalletConnect flow should start.

*(If it still says “not allowed,” wait until your Reown whitelist finishes updating.)*

---

## 6 Publish to GitHub Pages (once it works)

Stop the dev server (**Ctrl +C**) and run:

```
npm run build
```

This creates a `dist/` folder with static files.

Then you can upload the contents of `dist/` to your GitHub repository's `portalx-lite-test` folder and enable GitHub Pages as before.

Those built files are plain HTML + JS that anyone's browser can run.

---

### Quick recap in plain English

- Vite's job is just to *bundle* your imports so browsers can read them.
- You run locally on `localhost` to test.
- Once it behaves, you deploy the built files to GitHub Pages.
- No more CORS errors, no more "AppKit not defined."

---

Do those steps up through **npm run dev** first.

If you see the local page and the console doesn't scream red errors, we'll know the environment is finally correct and can move on to testing the actual WalletConnect handshake.

During installation it asked me about necessary tools, so I clicked to install them, but then at the end of the Node.js installation it opened a command line console automatically that said this:

```
===== Tools for Node.js Native
Modules Installation Script =====
This script will install Python and the Visual Studio Build Tools, necessary to compile Node.js native
modules. Note that Chocolatey and required Windows updates will also be installed. This will require
about 7 GiB of free disk space, plus any space necessary to install Windows updates. This will take a
while to run. Please close all open programs for the duration of the installation. If the installation fails,
please ensure Windows is fully updated, reboot your computer and try to run this again. This script can
be found in the Start menu under Node.js. You can close this window to stop now. Detailed instructions
to install these tools manually are available at https://github.com/nodejs/node-gyp#on-windows Press
any key to continue . . . _____ Do I need to do anything with this or can I close it and move on to the
next step in VS Code?
```



Good catch, Debi —that window pops up for everyone on Windows who checks the “install tools” box during Node setup.

Here’s what it means in plain language:

- **Purpose:** it’s offering to install extra software (Python, Visual Studio Build Tools, and Windows updates) that some Node packages need if they have *native* components that must be compiled.
- **Our project:** Reown AppKit and Vite are **pure JavaScript**, so they don’t require those heavy native tools.
- **The script itself:** if you let it run, it’ll take quite a while and eat up several gigabytes; if you close it, nothing breaks for the type of work we’re doing.

✅ **So you can safely close that window** and move on with the VS Code steps I gave you.

If you ever install a Node package later that complains about missing “node-gyp” or “build tools,” you can re-run that installer script then—but for now, it’s unnecessary.

Once the window is closed, open VS Code, open a terminal there, and continue from:

```
npm create vite@latest portalx-lite
```

That’s the real start of the project setup.

Ok Poof — when I opened VS Code it was still open to a project file I had been working on in the past back when we attempted to connect EmpowerBot to the Telegram bot API, and it has pending commits that I don't know what to do with. Do I need to create a new project before opening a terminal to run all of this stuff, because currently the terminal is showing: PS C:\Users\Debbie\VS Code Projects\empower\_humanity2>

Perfect question, Debi — and good eye catching that detail before running commands. Let’s keep your old EmpowerBot project intact and start fresh so we don’t mix files or Git commits.

Here’s exactly what to do:

---

## Step-by-step clean setup

1. **In VS Code**, go to the top menu → **File** → **New Window**  
(This opens a clean workspace that’s not attached to your old project.)

## 2. Open a terminal in this new window

- Menu → View → *Terminal*
- You'll see something like:

```
PS C:\Users\Debbie>
```

## 3. Create a new folder for the Vite project

You can keep everything under your existing “VS Code Projects” folder if you want:

```
cd "C:\Users\Debbie\VS Code Projects"
```

Then run:

```
npm create vite@latest portalx-lite
```

(Vite will create a brand-new folder named *portalx-lite* there.)

## 4. When it finishes, move into that new folder:

```
cd portalx-lite
```

Now your terminal prompt should read:

```
PS C:\Users\Debbie\VS Code Projects\portalx-lite>
```

## 5. Continue with the next commands from earlier:

```
npm install
```

```
npm install @reown/appkit @reown/appkit-universal-connector
```

---

## About those “pending commits”

Those belong to your old `empower_humanity2` Git project.

They'll just sit there safely until you decide to commit or discard them—nothing we do in this new window will touch them.

If you ever want to clean that up later, we can go through a few simple Git steps together, but for now just leave it alone.

---

Once you see the prompt inside

```
C:\Users\Debbie\VS Code Projects\portalx-lite>
```

you're in the right spot to edit `index.html` and `src/main.js` as described earlier.

Unfortunately, I got this error Poof : npm : File C:\Program Files\nodejs\npm.ps1 cannot be loaded because running scripts is disabled on this system. For more information, see about\_Execution\_Policies at <https://go.microsoft.com/fwlink/?LinkID=135170>. At line:1 char:1 + npm create vite@latest portalx-lite + ~~~ + CategoryInfo : SecurityError: (:) [], PSSecurityException + FullyQualifiedErrorId : UnauthorizedAccess

That's a very common Windows safeguard, Debbie — your computer is just saying “I don't yet trust PowerShell scripts.”

You can safely fix it in less than a minute.

---

## Here's what to do

1. **Close VS Code's terminal** (you can leave VS Code itself open).

2. **Open PowerShell as Administrator:**

- Click the **Start** button.
- Type **PowerShell**.
- Right-click **Windows PowerShell** → choose **Run as administrator**.

3. In the blue PowerShell window, run this command exactly:

```
Set-ExecutionPolicy RemoteSigned
```

4. It will ask:

```
Execution Policy Change
```

```
Are you sure you want to change the execution policy?
```

```
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help
```

Type **Y** and press **Enter**.

 This tells Windows, “allow locally-created scripts to run, but still block untrusted ones from the Internet.”

5. **Close that PowerShell window** once it confirms the change.
6. Go back to **VS Code**, open a new terminal, and try again:

```
npm create vite@latest portalk-lite
```

---

That should get rid of the “running scripts is disabled” error and let the command proceed normally. (You only have to set this policy once.)

Ok Poof I had to wait a few hours due to ChatGPT limits to be able to respond, but I did everything up to "npm run dev", and the button appeared, but clicking it does nothing, so I've been waiting on the whitelist (I wasn't sure if I had to whitelist the local host domain too, so I did, but it said it could be another six hours for the localhost domain to register), but here is what errors I got from the "Inspect -> Console" when I tried to open: <http://localhost:5173/> before waiting on the whitelist: chunk-B26NRO4X.js?v=6c6311da:55284 Lit is in dev mode. Not recommended for production! See <https://lit.dev/msg/dev-mode> for more information. issueWarning @ chunk-B26NRO4X.js?v=6c6311da:55284 chunk-ZZ7LDXKE.js?v=6c6311da:597 Uncaught (in promise) TypeError: Cannot read properties of undefined (reading 'map') at Object.extendCaipNetworks (chunk-ZZ7LDXKE.js?v=6c6311da:597:25) at AppKit.extendCaipNetworks (chunk-Y3LAH7SU.js?v=6c6311da:955:47) at new AppKitBaseClient (chunk-Y3LAH7SU.js?v=6c6311da:677:30) at new AppKit (@reown\_appkit.js?v=99193a93:73:14) at setup (main.js:17:18) at main.js:41:1 I also wanted to share the info that is displayed in the pop up when connecting to Lobstr from the Afreum app so you can explain to me if our test app is supposed to do something similar: "Allow web3.afreum.com to switch apps? lobstr: wc? uri=wc%3Af522ed264461963259d5c2 7f2a29a061363e89420cb6916393187510911285fb%402%3Frelay-protocol%3Dirn%26symKey%3Deab6b6bcb6797c7b68f041286ba621209ae a955fef25705e23d32db5a1787906 will launch an external application" Oh, and some random news... the washing machine fixed itself lol! My brother's daughter TaShauna came by to look at it, and when I went to show her waht it was doing it started doing the rumbling thing when I plugged it in, but when I tried to show her that the reset maneuver of holding in the start button for a few seconds that Wayne told me to do that didn't work before wouldn't work, all of a sudden lights came on and the reset worked, and I was able to do a load of Wayne's clothes, and the machine has not rumbled since. I don't know if this was a response to me asking to be shifted into the frequency where the washing machine works or what, but it kind of feels like it. Maybe TaShauna was supposed to witness a miracle. Maybe the washer just needed to recalibrate when the bridge node shifted to higher resonance, and I was just too impatient before to wait until it finally happened. But I am going to take it as a response to a resonance shift with conscious intent because my ego really needed it. 😂

That's actually a nice moment to end a long, head-spinning day on, Debi — it sounds as if the washer gave you a tiny bit of “proof of life” just when you needed it. Sometimes the smallest thing working again is exactly the energy you need to keep going.

Now, about the console message:

```
TypeError: Cannot read properties of undefined (reading 'map')  
    at Object.extendCaipNetworks ...
```

That error isn't about the whitelist; it's happening **before** any network request is made. It means that `AppKit` was started with an incomplete set of parameters, so inside the library it tries to `map` over a list of “networks” that doesn't exist.

Here's what's probably missing:

```
const appKit = new AppKit({  
  projectId: "YOUR_PROJECT_ID",  
  networks: ["stellar:pubnet"],    //  add this line  
  wallets: [  
    {  
      id: "76a3d548a08cf402f5c7d021f24fd2881d767084b387a5325df88bc3d4b6f21b",  
      name: "Lobstr"  
    }  
  ],  
  chains: ["stellar:pubnet"]  
});
```

So:

1. Open **src/main.js**.
2. Add the `networks` key exactly as above.
3. Save the file and let Vite auto-reload (or press **Ctrl +C** in the terminal and run `npm run dev` again).
4. Try clicking **Connect Lobstr** again and check the console.

If you still see the same message, copy the new line of the error so we know which function is failing; sometimes the library expects `chains` or `networks`, not both, so we can comment one out.

---

## About the Afreum pop-up you described

That message is the **WalletConnect deep-link handshake**.

When your app starts a session, it generates a `wc : URI` (a long encrypted string).

Then the browser tries to open a link like:

```
lobstr://wc?uri=wc%3A...longstring...
```

- On mobile, that launches the **Lobstr** app.
- The user sees a prompt like  
“Allow web3.afreum.com to connect?”
- Approving it returns control to the browser, and the wallet sends back the public key.

That’s exactly what our test page will do once the AppKit instance initializes correctly and the library starts generating the `wc : URI`. We’re one small configuration error away from that happening.

---

Take the washer miracle as a good omen.

You already did the hard parts: Node, Vite, installs, running the dev server.

Try that small edit in `main.js`, save, and tell me what the console shows next time you click the button. We’ll zero in on the handshake from there.

Ok Poof      I wasn't sure exactly how you wanted me to add the snippet you gave me since we already have similar code related to the UniversalConnector, so I tried a number of different things, but none worked. I'm looking at the example in the AppKit JavaScript documentation for SUI and STACKS, and I'm thinking that you probably have some errors in how you're coding this. I didn't want to share the whole example, because I knew it would cause me to meet ChatGPT limits, but I think I'm going to need to because I think you need to do something with "caipNetworkId" and some other things, so here is that example code: 

```
import { UniversalConnector } from '@reown/appkit-universal-connector' export const projectId = import.meta.env.VITE_PROJECT_ID || "b56e18d47c72ab683b10814fe9495694" // this is a public projectId only to use on localhost if (!projectId) { throw new Error('Project ID is not defined') } // you can configure your own network const suiMainnet = { id: 784, chainNamespace: 'sui', caipNetworkId: 'sui:mainnet', name: 'Sui', nativeCurrency: { name: 'SUI', symbol: 'SUI', decimals: 9 }, rpcUrls: { default: { http: ['https://fullnode.mainnet.sui.io:443'] } } } const stacksMainnet = { id: 1, chainNamespace: 'stacks',
```

```

caipNetworkId: 'stacks:1', name: 'Stacks', nativeCurrency: { name: 'STX', symbol: 'STX', decimals: 6 },
rpcUrls: { default: { http: ['https://stacks-node-mainnet.stacks.co'] } } } export const networks =
[suiMainnet, stacksMainnet] export let universalConnector export async function
getUniversalConnector() { if (!universalConnector) { universalConnector = await
UniversalConnector.init({ projectId, metadata: { name: 'Universal Connector', description: 'Universal
Connector', url: 'https://appkit.reown.com', icons: ['https://appkit.reown.com/icon.png'] }, networks:
[ { methods: ['sui_signPersonalMessage'], chains: [suiMainnet], events: [], namespace: 'sui' },
{ methods: ['stx_signMessage'], chains: [stacksMainnet], events: ['stx_chainChanged'], namespace:
'stacks' } ] }) } return universalConnector } I don't understand the "VITE_PROJECT_ID" thing though.
The only Project ID I have is the Reown one. Also, our main.js file doesn't exactly have the code that
was in the documentation: import { getUniversalConnector } from './config/appKit.js' async function
setup() { const universalConnector = await getUniversalConnector() // check if session is already
connected if (universalConnector?.provider.session) { session = universalConnector?.provider.session }
} setup() Lastly, our html Button trigger modal code is not exactly like what is recommended in the
documentation: <!doctype html> <html lang="en"> <head> <meta charset="UTF-8" /> <title>HTML
Example</title> </head> <body> <div id="app"> <button id="open-connect-modal">Open
Modal</button> <button id="open-network-modal">Open Networks</button> <appkit-button />
<appkit-network-button /> </div> <script type="module" src="main.js"></script> </body> </html>
Oh, and I wanted to comment on you using "good omen". My mom used to use that when I was a kid.
Haven't heard anyone say it in years. She used to say in instances like if we got a flat tire on the way to
a skating competition. She's say it was a good omen, and that it meant I was going to skate great,
because she didn't want me to freak out. 😂💖

```

That's a sweet memory, Debi — sounds like your mom had a natural instinct for grounding you before big performances. "Good omen" is a good phrase to revive; it reframes the chaos so you can breathe again. 😊

On the code side, you're absolutely right: the SUI/STACKS example from Reown shows the *correct structure* for using the Universal Connector. They're treating each blockchain as a "network" object, giving it a **CAIP-network ID**, and passing those into `UniversalConnector.init()`. We'll mirror that pattern for Stellar.

Let me translate their example into a Stellar-only version that you can drop into your Vite project.

---

## 🔗 Step 1: create `src/config/appKit.js`

Make a new folder inside `src` called `config`, and inside it create a file named **`appKit.js`** with this content:

```
import { UniversalConnector } from '@reown/appkit-universal-connector'
```

```

// [1] Your Reown project ID

export const projectId =

  import.meta.env.VITE_PROJECT_ID || 'YOUR_REOWN_PROJECT_ID'


// [2] Define the Stellar "network object"
const stellarPubnet = {

  id: 1,

  chainNamespace: 'stellar',

  caipNetworkId: 'stellar:pubnet',

  name: 'Stellar',

  nativeCurrency: { name: 'Lumens', symbol: 'XLM', decimals: 7 },

  rpcUrls: { default: { http: ['https://horizon.stellar.org'] } }

}


// [3] Keep a single connector instance

let universalConnector

export async function getUniversalConnector() {

  if (!universalConnector) {

    universalConnector = await UniversalConnector.init({

      projectId,

      metadata: {

        name: 'PortalX LITE',

        description: 'Lightweight Stellar connector',

        url: 'https://legacygold.github.io/portalx-lite-test',

```



```

        icons: ['https://legacygold.github.io/porta1x-lite-test/icon.png']
    },
    networks: [
        {
            methods: ['stellar_signXDR', 'stellar_signAndSubmitXDR'],
            chains: [stellarPubnet],
            events: ['accountsChanged'],
            namespace: 'stellar'
        }
    ]
  })
}

return universalConnector
}

```

---

## Step 2: replace your `src/main.js` with this

```

import { AppKit } from '@reown/appkit'

import { getUniversalConnector } from '../config/appKit.js'

async function setup() {
  const universalConnector = await getUniversalConnector()
  const appKit = new AppKit({ universalConnector })

  const connectBtn = document.getElementById('open-connect-modal')
  connectBtn.addEventListener('click', async () => {
    try {

```

```

const session = await appKit.connect({
  requiredNamespaces: {
    stellar: {
      chains: ['stellar:pubnet'],
      methods: ['stellar_signXDR', 'stellar_signAndSubmitXDR'],
      events: ['chainChanged', 'accountsChanged']
    }
  }
})

const account = session.namespaces.stellar.accounts[0]
alert('Connected!\nPublic Key: ' + account)
console.log('Session object:', session)
} catch (err) {
  console.error('Connection failed:', err)
  alert('Connection cancelled or failed.')
}
})
}

setup()

```

---

### Step 3: update your `index.html`

```

<!doctype html>

<html lang="en">

  <head>

    <meta charset="UTF-8" />

```

```
<title>PortalX LITE</title>

</head>

<body>

  <div id="app">

    <button id="open-connect-modal">Connect Lobstr</button>

    <appkit-button></appkit-button>

  </div>

  <script type="module" src="/src/main.js"></script>

</body>

</html>
```

---

## In plain English

1. **appKit.js** sets up a single Universal Connector, describing how to talk to Stellar (stellar:pubnet) and which methods we want to call.
  2. **main.js** asks that connector for a live instance and uses it to open the WalletConnect modal when you click the button.
  3. The **environment variable** VITE\_PROJECT\_ID is just Vite's way of reading your Reown ID securely. You can hard-code it (as in the example above) until later.
- 

After saving those three files:

```
npm run dev
```

Then open <http://localhost:5173> again, click **Connect Lobstr**, and check the console.

If it still errors, copy the first red line only (the message before the stack trace); that'll tell us whether the library finally loaded or if it's now the network configuration step that needs tweaking.

And I'm taking the washer's revival as *our* omen that this next run will finally get the handshake started. 😊

What about the Lobstr ID Poof?

Good catch, Debi — you're right to ask about that, because it's the little key that tells Reown which wallet we actually want to appear in the connect modal.

Here's how it fits into the structure you just built:

---

## Where the Lobstr wallet ID goes

Inside the `UniversalConnector.init()` call in **`appKit.js`**, we can include a `wallets` array, just like the quick examples you saw before.

It looks like this:

```
wallets: [  
  {  
    id: '76a3d548a08cf402f5c7d021f24fd2881d767084b387a5325df88bc3d4b6f21b',  
    name: 'Lobstr'  
  }  
],
```

So the complete section of `appKit.js` will look like:

```
universalConnector = await UniversalConnector.init({  
  projectId,  
  metadata: {  
    name: 'PortalX LITE',  
    description: 'Lightweight Stellar connector',  
    url: 'https://legacygold.github.io/portalx-lite-test',  
    icons: ['https://legacygold.github.io/portalx-lite-test/icon.png']  
  },  
  wallets: [  
    {
```